

INFORMACJE OGÓLNE

Celem projektu jest stworzenie interaktywnej gry edukacyjno-rozrywkowej, w której gracz eksploruje planszę, zbiera przedmioty, rozwiązuje zagadki logiczne oraz unika zagrożeń, aby ukończyć kolejne poziomy. Gra łączy elementy: zręcznościowe, logiczne, eksploracyjne, strategiczne.

FABUŁA

Akcja rozgrywa się w tajemniczym kompleksie podziemnych labiryntów znajdujących się pod ruinami dawnego miasta zaginionej cywilizacji. Według legend w głębi labiryntu ukryto pradawny artefakt zwany **Oko Smoka**, posiadający moc kontrolowania energii całego świata.

Przez wiele lat wejście do podziemi pozostawało zamknięte, jednak w wyniku zmian klimatycznych korytarze ponownie stały się dostępne. W tym samym czasie zaczęły pojawiać się dziwne zjawiska:

- zaniki energii,
- tajemnicze światła,
- agresywne stworzenia wydostające się nocą z ruin.

Główny bohater — młody odkrywca i badacz starożytnych technologii — otrzymuje zadanie odnalezienia źródła zagrożenia i zabezpieczenia artefaktu, zanim trafi on w niepowołane ręce.

ZAKRES PROJEKTU

Projekt obejmuje:

- stworzenie silnika w JavaScript
- implementację planszy typu grid
- system sterowania postacią
- system poziomów
- system przeciwników i pułapek
- mechanikę zbierania przedmiotów
- zagadki logiczne
- interfejs użytkownika
- efekty dźwiękowe
- dokumentację projektu
- testowanie i wdrożenie aplikacji

TECHNOLOGIE

1. **Frontend:** HTML5, CSS3, JavaScript ES6+
2. **Renderowanie gry:** HTML5 Canvas
3. **Narzędzia wspomagające:** Git / GitHub, Trello (Kanban)
4. **Środowisko uruchomieniowe:** przeglądarka internetowa: Google Chrome, Microsoft Edge, FireFox

ARCHITEKTURA SYSTEMU

1. Struktura modułów

Moduł	Opis
Engine	główna pętla gry
Player	logika gracza
Maze	obsługa planszy
Items	obsługa przedmiotów
Enemies	logika przeciwników
Puzzles	obsługa zagadek
UI	interfejs użytkownika
Levels	zarządzanie poziomami

2. Struktura katalogów

```

/project
|
├─ index.html
├─ /css
|   └─ style.css
|
├─ /js
|   └─ game.js
|   └─ engine.js
|   └─ player.js
|   └─ maze.js
|   └─ items.js
|   └─ enemies.js
|   └─ puzzles.js
|   └─ ui.js
|   └─ levels.js
|
├─ /assets
|   └─ /images

```

WYMAGANIA FUNKCYJONALNE

1. Sterowanie postacią – gracz steruje postacią za pomocą klawiatury

- Funkcjonalności:
 - ruch WASD lub strzałkami
 - animacja ruchu
 - blokada ruchu przez ściany
 - płynne przemieszczanie postaci

2. Mechanika labiryntu – gracz porusza się po planszy w celu odnalezienia wyjścia

- **Funkcjonalności:**
 - plansza typu grid
 - ściany i przejścia
 - punkt startowy
 - punkt końcowy
 - wykrywanie kolizji
 - przejście do kolejnego poziomu

3. System przedmiotów

- **Typy przedmiotów:**
 - apteczki (+HP)
 - klucze
 - przedmioty fabularne
 - aktywatory zagadek
- **Funkcjonalności:**
 - zbieranie przedmiotów
 - aktualizacja ekwipunku
 - usuwanie przedmiotu po zebraniu

4. System zagadek - zagadki aktywowane są po wejściu w odpowiedni obszar lub po zebraniu przedmiotu

- **Typy zagadek**
 - pytania logiczne
 - kody dostępu
 - sekwencje znaków
 - zagadki pamięciowe
- **Funkcjonalności**
 - blokada przejścia
 - sprawdzanie poprawności odpowiedzi
 - komunikaty o wyniku

5. System zagrożeń

- **Rodzaje zagrożeń**
 - pułapki
 - przeciwnicy patrolujący mapę
- **Funkcjonalności**
 - kolizje z przeciwnikami
 - odejmowanie punktów życia
 - ekran Game Over po utracie HP

6. System poziomów – gra posiada 3 poziomy

- **Rozwój trudności**
 - większe mapy
 - większa liczba przeciwników
 - bardziej złożone zagadki

INTERFEJS UŻYTKOWNIKA

1. Elementu HUD - informacje wyświetlane na ekranie

- pasek zdrowia
- numer poziomu
- liczba zebranych przedmiotów
- komunikaty systemowe

2. Ekran systemowe

- ekran startowy
- ekran zwycięstwa
- ekran porażki
- ekran przejścia poziomu

WYMAGANIA NIEFUNKCJONALNE

1. Wydajność

- minimum 60 FPS
- płynna animacja postaci
- szybkie ładowanie poziomów

2. Responsywność

- obsługa różnych rozdzielczości
- poprawne skalowanie Canvas

3. Użyteczność

- intuicyjna obsługa
- czytelny interfejs
- spójna stylistyka graficzna

4. Bezpieczeństwo

- brak przechowywania danych użytkownika
- lokalne działanie aplikacji

5. Rozszerzalność

- modularna architektura kodu
- możliwość dodawania nowych poziomów i przeciwników

TESTOWANIE

1. Rodzaje testów

- testy funkcjonalne
- testy wydajnościowe
- testy interfejsu użytkownika
- testy kompatybilności przeglądarek

2. Kryteria akceptacji

- brak błędów krytycznych
- poprawna obsługa poziomów

SPECYFIKACJA PROJEKTU – LABIRYNT

- poprawne działanie zagadek i kolizji
- stabilność działania gry